LABORATORIO N° 3a

"Trabajar con punto fijo en el procesador"

FACULTAD DE INGENIERÍA - UNAM

CARRERA:

Ingeniería en Computación

MATERIA:

IC221 - Arquitectura de computadoras

DOCENTES:

Dr. Ing. Roberto E. Carballo

Dr. Ing. Aldo J. Benitez.

Ing. Roberto N. Schuster

Ing. Patricia V. Feltan

Ing. Gustavo O. Rossler

Ing. Eduardo Díaz

Sr. Joaquín Nahuel Giménez

Sr. Joaquín Zuk

ALUMNOS:

Meretz Eliana Marife

Meretz Mariana Belen

GRUPO: 30

ÍNDICE

1. INTRODUCCIÓN 3
   1.1 ¿Qué es un número en punto fijo? 3
   1.2 La notación Qm.n 3
   1.3 Palabras clave 4
   1.4 Objetivos 4
2. PROBLEMA N°1: PUNTO FIJO EN LENGUAJE C 4
   2.1 DESARROLLO EN C 4
      2.1.1 Actividad 1: ingreso de los números y breakpoints 5
      2.1.2 Actividad 2: pasar cada número a punto fijo (casting) 5
      2.1.3 Actividad 3: cambiar de formato, sumar y restar 5
      2.1.4 Actividad 4: multiplicación 6
      2.1.5 Actividad 5: división 6
   2.2 RESULTADOS EN C 7
   2.3 DISCUSIÓN DE LOS RESULTADOS EN C 7
3. PROBLEMA N°2: PUNTO FIJO EN ASSEMBLY DE LA LC-3 8
   3.1 DESARROLLO EN ASSEMBLY 8
      3.1.1 Qué puede (y qué no) hacer la LC-3 8
      3.1.2 Datos en memoria y estructura del programa 8
      3.1.3 Conversión entre formatos 8
      3.1.4 Suma y resta 9
      3.1.5 Multiplicación 9
   3.2 RESULTADOS EN ASSEMBLY 9
   3.3 DISCUSIÓN DE LOS RESULTADOS EN ASSEMBLY 10
4. USO DE LA INTELIGENCIA ARTIFICIAL 10
5. CONCLUSIONES 10
6. REFERENCIAS BIBLIOGRÁFICAS 11

1. INTRODUCCIÓN

Este informe explica, con palabras sencillas, cómo una computadora puede trabajar con números que tienen coma (como 3,14 o −5,75) sin usar los complicados circuitos de los números “flotantes”. La técnica se llama **punto fijo** y se practicó en dos lenguajes: C (un lenguaje de alto nivel) y el assembly de la LC-3, un procesador sencillo que se usa para aprender cómo funciona una computadora por dentro [1].

1.1. ¿Qué es un número en punto fijo?

Una computadora guarda todo como **bits**, es decir, ceros y unos. Con ellos es muy natural escribir números enteros (1, 25, −8), pero no existe ningún bit especial para “la coma” de un número como 3,14. Hay dos formas de resolver este problema. La primera, llamada **punto flotante** (los tipos `float` y `double` de C), guarda junto al número la información de dónde está la coma; por eso la coma “flota”. Es cómoda, pero necesita circuitos más grandes y complejos. La segunda es el **punto fijo**: se guardan solo números enteros y todos se ponen de acuerdo, de antemano, en dónde está la coma. La coma no se guarda en ningún lugar; está “imaginada” y siempre en el mismo sitio [2].

Una analogía cotidiana son los precios expresados en centavos. El precio de $3,14 se guarda como 314 centavos, un número entero. Quien lee ese número sabe, sin que nadie se lo diga, que hay que dividirlo por 100 para recuperar los pesos. En la computadora ocurre lo mismo, pero en lugar de 100 se usa una potencia de 2 (por ejemplo 2^12 = 4096), porque multiplicar o dividir por una potencia de 2 equivale a correr los bits hacia un lado, algo que el procesador hace muy rápido. Las dos fórmulas básicas son:

- **Para guardar:** valor guardado = valor real × 2^n (y se redondea al entero más cercano).
- **Para leer:** valor real = valor guardado ÷ 2^n.

Por ejemplo, con n = 12: 3,14159 × 4096 = 12868,1…, que se guarda como el entero 12868. Al leerlo, 12868 ÷ 4096 = 3,1416015…, un valor muy parecido al original pero no idéntico. Esa pequeña diferencia es el precio que se paga por usar enteros.

1.2. La notación Qm.n

Para indicar dónde está la coma imaginaria se usa la notación **Qm.n**. En ella, el número se guarda en 16 bits con: 1 bit de signo (para distinguir positivos de negativos), **m** bits para la parte entera y **n** bits para la parte fraccionaria, de modo que m + n = 15. Cuantos más bits fraccionarios hay, más pequeña es la diferencia entre dos números vecinos (la **resolución**, igual a 1/2^n), pero menor es el rango de valores que se pueden escribir. Es un compromiso: más precisión o más alcance. La Tabla 1 muestra los dos formatos usados en este laboratorio y la Figura 1 muestra cómo se ven sus bits.

Tabla 1: Formatos de punto fijo usados en el laboratorio

Formato	Bits enteros	Bits fraccionarios	Rango	Resolución	El 1,0 se guarda como
Q3.12	3	12	de −8 a 7,99976	1/4096 ≈ 0,000244	4096
Q7.8	7	8	de −128 a 127,996	1/256 ≈ 0,0039	256

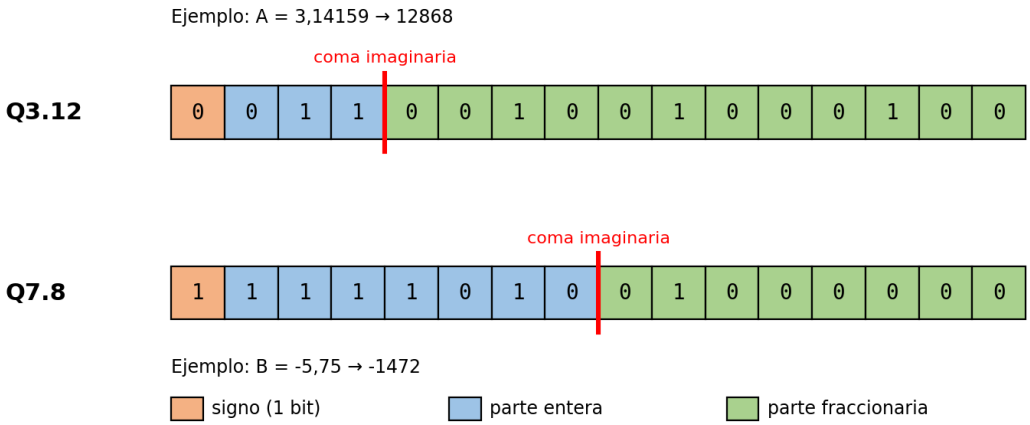


Figura 1: Los 16 bits de un número en Q3.12 y de otro en Q7.8. La línea roja marca la coma imaginaria; no ocupa ningún bit.

1.3. Palabras clave

La Tabla 2 reúne los términos técnicos que aparecen en el informe, para poder consultarlos cuando haga falta.

Tabla 2: Glosario de términos usados

Término	Qué significa
Bit	La unidad mínima de información: vale 0 o 1. Aquí cada número ocupa 16 bits.
Desplazamiento (shift)	Correr todos los bits hacia la izquierda (equivale a multiplicar por 2 por cada lugar) o hacia la derecha (equivale a dividir por 2).
Redondeo	Elegir el valor representable más cercano en vez de cortar los decimales sobrantes.
Desborde	Cuando el resultado no entra en los 16 bits disponibles y se obtendría un número equivocado.
Saturación	Ante un desborde, dejar el resultado en el mayor (o menor) valor posible en vez de uno equivocado.
Casting	Convertir un dato de un tipo a otro, por ejemplo de `double` a entero de 16 bits.
Breakpoint	Marca que detiene el programa en una línea para poder revisarlo paso a paso.
Variable vigilada (watch)	Variable cuyo valor se muestra en el entorno de desarrollo mientras el programa está detenido.

1.4. Objetivos

Los objetivos de este trabajo, planteados en la guía de laboratorio [3], son:

- Representar números con coma en dos formatos de punto fijo distintos (Q3.12 y Q7.8), en C y en la LC-3.
- Convertir un número de un formato Q a otro para poder sumar y restar.
- Multiplicar dos números en punto fijo y ajustar el resultado al formato elegido.
- Dividir dos números conservando la mayor resolución posible (en C).
- Depurar programas en C y en assembly con breakpoints y variables vigiladas.
- Analizar con criterio el código generado por inteligencia artificial.

2. PROBLEMA N°1: PUNTO FIJO EN LENGUAJE C

2.1. DESARROLLO EN C

Se escribió un programa en C (archivo `lab3a_punto_fijo.c`) que pide dos números con coma y realiza las cinco actividades pedidas en la guía. Se usó el tipo `double` para los números con coma y el tipo `int16_t` (entero de 16 bits con signo) para los números en punto fijo. El número A se guardó en formato Q3.12 y el número B en formato Q7.8, de modo que cada uno quedó en un formato diferente, como pedía la consigna. Como datos de prueba se usaron **A = 3,14159** y **B = −5,75**.

2.1.1. Actividad 1: ingreso de los números y breakpoints

Los números se leyeron con `scanf("%lf", ...)` y se guardaron en las variables `a_real` y `b_real`. Antes de seguir, el programa comprobó que cada valor entrara en el rango de su formato (A entre −8 y 7,99976; B entre −128 y 127,996); si no entraba, se avisó con un mensaje. Se marcaron cinco breakpoints en el código, uno por actividad, y en cada uno se agregaron a las variables vigiladas los valores de la Tabla 3. Al detener el programa en cada punto, los valores se observaron en el entorno de desarrollo (no en la consola).

Tabla 3: Breakpoints del programa en C y valores observados con A = 3,14159 y B = -5,75

N°	Dónde se coloca	Variables vigiladas	Valores observados
1	Después de leer A y B	a_real, b_real	3,14159 y −5,75
2	Después del casting	a_q, b_q	12868 (0x3244) y −1472 (0xFA40)
3	Después de convertir, sumar y restar	a_en_q78, b_en_q312, suma_q, resta_q	804, −23552, −668 y 2276
4	Después de multiplicar	prod_crudo, prod_q	−18941696 y −4624
5	Después de dividir	div_q, n_div	−17903 y 15

2.1.2. Actividad 2: pasar cada número a punto fijo (casting)

Cada número se multiplicó por 2^n (n = 12 para A, n = 8 para B) y luego se hizo el casting a `int16_t`. Como el casting corta los decimales en lugar de redondear, se sumó 0,5 antes de convertir (o se restó, si el número era negativo); así el resultado queda en el entero más cercano.

```
static int16_t float_a_q(double x, int n)
{
    double escalado = x * (double)(1 << n);      /* x * 2^n */
    return (int16_t)(escalado >= 0.0 ? escalado + 0.5
                                     : escalado - 0.5);
}
```

Para A: 3,14159 × 4096 = 12868,1 → se guardó 12868 (0x3244). Para B: −5,75 × 256 = −1472 exactamente → se guardó −1472 (0xFA40), sin error porque 0,75 se puede escribir justo con bits (0,75 = 1/2 + 1/4).

2.1.3. Actividad 3: cambiar de formato, sumar y restar

Dos números en punto fijo solo se pueden sumar o restar si tienen la coma imaginaria en el mismo lugar. Es como sumar pesos con centavos: 1 peso más 50 centavos no es “1 + 50”. En este caso, el valor 1,0 se guarda como 4096 en Q3.12 pero como 256 en Q7.8; si se sumaran los enteros directamente, 4096 + 256 = 4352 no tendría ningún significado.

Para compatibilizar los formatos se usó un **desplazamiento de bits**. Para pasar de Q3.12 a Q7.8 se pierden 4 bits fraccionarios (12 - 8 = 4), lo que equivale a dividir por 16: se desplazan los bits 4 lugares a la **derecha**. Antes de desplazar se suma 8 (la mitad de 16) para que el resultado se redondee en lugar de cortarse. Para el camino inverso, de Q7.8 a Q3.12, se ganan 4 bits: se desplaza 4 lugares a la **izquierda** (multiplicar por 16); aquí puede haber desborde si el número vale 8 o más, por eso se satura. Para sumar y restar se eligió el formato común **Q7.8**, porque tiene más rango y evita desbordes.

```
if (n_dest < n_orig) {                  /* se pierde resolución  */
    int k = n_orig - n_dest;
    mag = (mag + (1 << (k - 1))) >> k;  /* desplazar a la derecha */
} else if (n_dest > n_orig) {           /* se gana resolución    */
    int k = n_dest - n_orig;
    mag = mag << k;                     /* desplazar a la izquierda */
    if (mag > 32767) { mag = 32767; *saturo = 1; }
}
```

Con los datos de prueba: A en Q7.8 = (12868 + 8) >> 4 = 804, es decir 804 ÷ 256 = 3,140625. Luego, con ambos números en Q7.8, la suma y la resta son operaciones normales entre enteros (804 + (−1472) = −668 y 804 - (−1472) = 2276). El resultado se calculó en 32 bits y se limitó al rango de 16 bits (saturación) por si hubiera desborde.

2.1.4. Actividad 4: multiplicación

Al multiplicar los enteros guardados se obtiene un resultado con **más bits fraccionarios**: (A × 2^12) × (B × 2^8) = A × B × 2^20. Es decir, el producto queda con 12 + 8 = 20 bits fraccionarios y no entra en 16 bits, por lo que se calculó en 32 bits. Para volver al formato elegido, Q7.8, hay que descartar 20 - 8 = 12 bits: se hizo un **desplazamiento a la derecha de 12 lugares** (dividir por 4096), sumando antes 2^11 = 2048 para redondear. La Figura 2 resume el proceso.

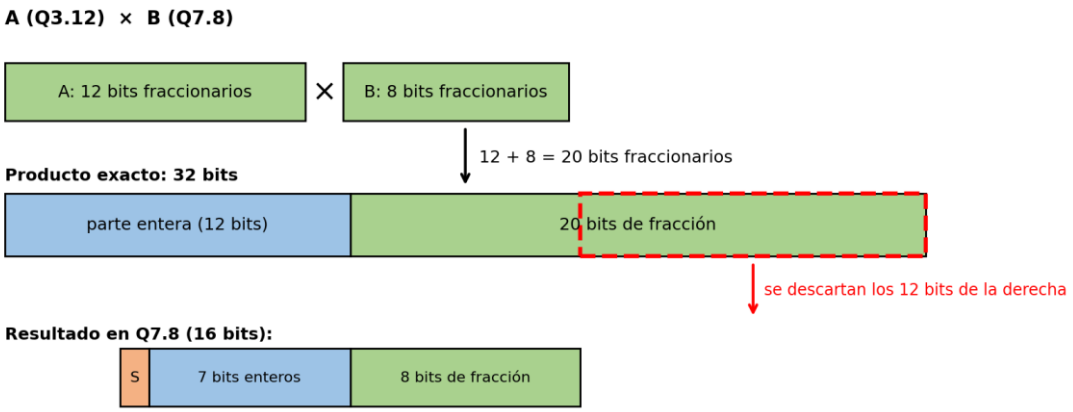


Figura 2: Al multiplicar un número Q3.12 por uno Q7.8 el producto tiene 20 bits fraccionarios; para llevarlo a Q7.8 se descartan 12 bits.

```
int64_t prod = (int64_t)ma * mb;          /* producto exacto           */
int k = (n_a + n_b) - n_res;              /* 12 + 8 - 8 = 12           */
int64_t r = (prod + ((int64_t)1 << (k-1))) >> k;   /* ajuste a Q7.8  */
```

Con los datos de prueba: 12868 × (−1472) = −18941696 (producto exacto). Se descartan 12 bits: (18941696 + 2048) >> 12 = 4624, y con el signo negativo queda −4624. Al leerlo, −4624 ÷ 256 = −18,0625.

2.1.5. Actividad 5: división

Si simplemente se dividen los dos enteros guardados, el cociente tiene n_a - n_b = 12 - 8 = 4 bits fraccionarios y se pierde casi toda la parte decimal. Con los datos de prueba, 12868 ÷ (−1472) da −8 (el entero cortado), que con 4 bits fraccionarios vale −8/16 = −0,5, cuando el valor correcto es −0,546. Para evitarlo, antes de dividir se **agranda el numerador** desplazándolo a la izquierda (se usa un entero de 64 bits para que no desborde): así el cociente conserva n bits fraccionarios, con n a elección.

Para obtener la **máxima resolución posible**, el programa prueba valores de n desde 15 hacia abajo y se queda con el mayor n para el cual el cociente todavía entra en 16 bits con signo (hasta 32767). El resultado queda en formato **Q(15-n).n**. Así el formato no se elige a ciegas, sino según el tamaño real del resultado: si el cociente es chico, sobran bits para la fracción; si es grande, hacen falta más bits enteros.

```
for (int n = 15; n >= 0; n--) {
    int64 t num = ma << (n + n b);        /* numerador agrandado */
    int64_t den = mb << n_a;
    int64_t q = (2 * num + den) / (2 * den);   /* con redondeo   */
    if (q <= 32767) { *n res = n; return ...; } /* entra: usar n */
}
```

Con A = 3,14159 y B = −5,75 el cociente vale −0,546, menor que 1 en valor absoluto, así que alcanzó con n = 15 (formato **Q0.15**, resolución 1/32768 ≈ 0,00003). Con otro par de prueba, A = 1,5 y B = 0,25, el cociente vale 6: con n = 13 daría 6 × 8192 = 49152, que no entra en 16 bits, mientras que con n = 12 da 24576, que sí entra; por eso el formato elegido fue **Q3.12**.

2.2. RESULTADOS EN C

La Tabla 4 muestra los resultados que arrojó el programa con A = 3,14159 y B = −5,75, comparados con el cálculo hecho con `double` (valor esperado). La última columna es el error: la diferencia entre ambos.

Tabla 4: Resultados del programa en C con A = 3,14159 y B = -5,75

Valor	Entero	Hex	Valor real leído	Esperado (double)	Error
A en Q3.12	12868	0x3244	3,14160156	3,14159	0,000012
B en Q7.8	−1472	0xFA40	−5,75	−5,75	0
A en Q7.8	804	0x0324	3,140625	3,14159	0,000965
B en Q3.12	−23552	0xA400	−5,75	−5,75	0
A + B (Q7.8)	−668	0xFD64	−2,609375	−2,60841	0,000965
A - B (Q7.8)	2276	0x08E4	8,890625	8,89159	0,000965
A × B exacto (Q10.20)	−18941696	0xFEDEF900	−18,06420898	−18,0641425	0,000066
A × B (Q7.8)	−4624	0xEDF0	−18,0625	−18,0641425	0,001642
A ÷ B (Q0.15)	−17903	0xBA11	−0,5463562	−0,5463635	0,000007

Con A = 1,5 y B = 0,25, que son números que se pueden escribir exactamente en binario, todos los errores fueron cero: la suma dio 1,75, la resta 1,25, el producto 0,375 y la división 6,0.

2.3. DISCUSIÓN DE LOS RESULTADOS EN C

Los resultados confirman que el punto fijo da valores muy cercanos a los reales, pero no exactos, y que el error siempre se explica por la resolución del formato. Conviene separar de dónde viene cada diferencia:

- **Suma y resta:** el error de 0,000965 no lo produce la suma, sino la conversión de A a Q7.8, que descartó 4 bits (3,14160156 pasó a 3,140625). Sumar o restar enteros es exacto; lo que "cuesta" es perder bits al cambiar de formato.
- **Multiplicación:** el producto exacto tiene 20 bits fraccionarios y casi no tiene error (0,000066, que viene del redondeo inicial de A). Al ajustarlo a Q7.8 el error subió a 0,0016, que es menor que la mitad de la resolución de Q7.8 (0,0039 ÷ 2 ≈ 0,00195), como se espera de un redondeo al más cercano.
- **División:** el error es muy pequeño (0,000007) porque el resultado cabía en un formato con 15 bits fraccionarios. Elegir el formato según el tamaño del cociente permitió aprovechar casi toda la resolución disponible.

También se observa que los formatos Q no "avisan" cuando algo no entra: si un resultado supera el rango, el programa tiene que detectarlo y saturar. Por eso el código incluye banderas de saturación en cada operación.

3. PROBLEMA N°2: PUNTO FIJO EN ASSEMBLY DE LA LC-3

3.1. DESARROLLO EN ASSEMBLY

En esta parte se implementaron los puntos 3 y 4 de la guía (conversión de formato con suma y resta, y multiplicación) directamente en assembly de la LC-3 (archivo `lab3a_puntos_3_y_4.asm`). Los números no se ingresan por teclado: se escribieron en la memoria ya convertidos a su formato Q, como pedía la consigna.

3.1.1. Qué puede (y qué no) hacer la LC-3

La LC-3 es un procesador intencionalmente simple. Tiene ocho registros de 16 bits (R0 a R7) y muy pocas instrucciones: las usadas aquí son `ADD`, `AND`, `NOT` (operaciones), `BR` (saltos según el resultado anterior), `LD` y `ST` (leer y guardar en memoria), `JSR` y `RET` (llamar a una subrutina y volver). Lo más importante es lo que **no** tiene: no hay instrucción de multiplicar, no hay desplazamientos de bits y no hay una bandera de "acarreo" que avise cuando una suma se pasa de 16 bits. Todo eso hubo que construirlo con las pocas instrucciones disponibles. El truco más usado fue desplazar a la izquierda con `ADD R0, R0, R0`, que suma un número consigo mismo, es decir, lo multiplica por 2.

3.1.2. Datos en memoria y estructura del programa

Los datos se declararon con `.FILL`, que escribe un valor en una posición de memoria. A = 3,14159 en Q3.12 vale 12868 y B = −5,75 en Q7.8 vale −1472. Los resultados se reservaron con `.BLKW`.

```
A_Q12   .FILL #12868     ; 3,14159 en Q3.12 (3,14159 x 4096)
B_Q8    .FILL #-1472     ; -5,75   en Q7.8  (-5,75 x 256)
A_EN_Q8 .BLKW #1         ; aquí quedará A convertido a Q7.8
```

El programa principal solo carga los datos en los registros R0 y R1, llama a una subrutina y guarda el resultado en memoria. Cada subrutina recibe sus datos en R0 y R1, devuelve el resultado en R0 y restaura los demás registros que haya usado, para no molestar al resto del programa (Tabla 5).

Tabla 5: Subrutinas del programa en assembly

Subrutina	Qué hace
Q12_A_Q8	Pasa un número de Q3.12 a Q7.8 (desplazamiento a la derecha de 4, con redondeo).
Q8_A_Q12	Pasa un número de Q7.8 a Q3.12 (desplazamiento a la izquierda de 4; satura si no entra).
SUMA_Q8	Suma dos números del mismo formato, detectando desborde.
RESTA_Q8	Resta dos números del mismo formato, detectando desborde.
MUL_Q12_Q8	Multiplica A (Q3.12) por B (Q7.8) y devuelve el producto en Q7.8.
MUL32	Auxiliar de la anterior: multiplica con resultado de hasta 30 bits usando dos mitades de 15 bits.

3.1.3. Conversión entre formatos

De Q7.8 a Q3.12 es la más fácil: se trabaja con el valor sin signo, se lo multiplica por 16 con cuatro sumas consigo mismo y se repone el signo. Antes se comprueba que el valor absoluto sea menor que 2048 (que corresponde a 8,0 en Q7.8); si no lo es, no cabe en Q3.12 y se devuelve el máximo, 32767.

```
ADD  R0, R0, R0     ; R0 = R0 x 2
ADD  R0, R0, R0     ; R0 = R0 x 4
ADD  R0, R0, R0     ; R0 = R0 x 8
ADD  R0, R0, R0     ; R0 = R0 x 16   (desplazar 4 a la izquierda)
```

De Q3.12 a Q7.8 es más difícil, porque hay que desplazar a la **derecha** y la LC-3 no puede hacerlo. La solución fue “reconstruir” el número: primero se suma 8 al valor absoluto (redondeo); luego se recorren los 12 bits más altos, de izquierda a derecha, y se van acumulando en otro registro desplazándolo a la izquierda en cada vuelta. El resultado son justamente esos 12 bits altos, que es lo mismo que haber desplazado 4 lugares a la derecha.

3.1.4. Suma y resta

Se suma (o se resta) en 16 bits. Como no hay bandera de acarreo, el desborde se detecta mirando los signos: solo puede haber desborde al sumar si ambos números tienen el mismo signo y el resultado tiene el signo contrario (por ejemplo, dos positivos que dan negativo). En ese caso se devuelve 32767 o −32768, según corresponda. La resta se hizo como A + (-B), con la regla análoga.

3.1.5. Multiplicación

Como la LC-3 no multiplica, se usó el método de “multiplicar a mano”, el de las cuentas de la escuela: por cada bit en 1 del multiplicador, se suma el multiplicando desplazado a la posición de ese bit. Se trabajó con los valores absolutos y se contó cuántos números eran negativos: si era un número impar, el resultado se vuelve negativo al final.

Un problema es que el producto puede llegar a 30 bits y los registros tienen 16. Se resolvió guardando el acumulador en **dos mitades de 15 bits**, una alta (`hi`) y otra baja (`lo`), de modo que el valor es hi × 2^15 + lo. Cuando la suma de la mitad baja se pasa de 15 bits, el bit sobrante aparece en el bit 15 del registro (que lo hace ver como negativo); entonces se lo borra y se suma 1 a la mitad alta. Es la forma de reemplazar el acarreo que la LC-3 no tiene.

```
M32_LP  AND  R5, R1, R4      ; ¿el bit actual del multiplicador es 1?
        BRz  M32_NA
        ADD  R2, R2, R6      ; acc_lo += a_lo
        BRzp M32_NC
        AND  R2, R2, R7      ; acarreo: limpiar el bit 15 ...
        ADD  R3, R3, #1      ; ... y sumarlo a acc_hi
M32_NC  ADD  R3, R3, R0      ; acc_hi += a_hi
M32_NA  ADD  R6, R6, R6      ; multiplicando <<= 1  (sigue el próximo bit)
```

Para el ajuste a Q7.8 hay que desplazar 12 lugares a la derecha, y como (hi × 2^15 + lo) ÷ 2^12 = hi × 8 + lo ÷ 4096, se calculó hi × 8 con tres sumas consigo mismo, y lo ÷ 4096 restando 4096 una y otra vez hasta que no se pueda (a lo sumo 7 veces). El valor inicial de `lo` es 2048, que cumple el mismo papel de redondeo que en C. Si hi llega a 4096 o más, el producto no entra en Q7.8 y se devuelve 32767 (saturación).

3.2. RESULTADOS EN ASSEMBLY

El programa se ejecutó con los mismos datos de prueba que en C (A = 3,14159 y B = −5,75). Los valores finales de las cinco posiciones de resultado se muestran en la Tabla 6. La ejecución completa fue de 351 instrucciones y los cinco valores coincidieron exactamente con los obtenidos en C.

Tabla 6: Contenido de la memoria de la LC-3 al terminar el programa

Etiqueta	Decimal	Hexadecimal	Valor real leído	Qué es
A_EN_Q8	804	x0324	3,140625	A convertido a Q7.8
B_EN_Q12	−23552	xA400	−5,75	B convertido a Q3.12
SUMA	−668	xFD64	−2,609375	A + B en Q7.8
RESTA	2276	x08E4	8,890625	A - B en Q7.8
PRODUCTO	−4624	xEDF0	−18,0625	A × B en Q7.8

Además se probaron otros pares de valores: dos negativos (−2,5 y −3,0, con producto 7,5), valores iguales (7,5 y 7,5), números muy pequeños (0,001 y 0,001) y casos que no entran en el formato (7,9 y 100). En todos los casos el resultado coincidió con el cálculo esperado y, cuando correspondía, el programa saturó en 32767 o −32767 en lugar de devolver un número equivocado.

3.3. DISCUSIÓN DE LOS RESULTADOS EN ASSEMBLY

Que C y assembly den exactamente los mismos números muestra que el punto fijo es una idea independiente del lenguaje: lo que cambia es cuánto trabajo tiene que hacer el programador. Lo que en C se escribe con `>>` o con el operador `*`, en la LC-3 requirió bucles, sumas repetidas y una revisión cuidadosa de los signos. Esto ayuda a entender por qué los procesadores modernos incluyen desplazadores, multiplicadores y banderas de acarreo en el hardware.

Un caso especial es el valor −32768, que no tiene opuesto positivo en 16 bits; el programa lo trata como −32767 al multiplicar. Como limitación de este trabajo, el assembly se verificó con un emulador sencillo de la LC-3 escrito para esta comprobación; conviene repetir la ejecución en el simulador oficial de la cátedra para observar los registros y la memoria con breakpoints, como pide la guía.

4. CONCLUSIONES

El punto fijo permite trabajar con números con coma usando solo enteros. Alcanza con acordar en qué lugar está la coma (formato Qm.n): guardar es multiplicar por 2^n y leer es dividir por 2^n. El costo es un pequeño error de redondeo, que nunca superó la mitad de la resolución del formato.

Cada formato implica un compromiso entre precisión y rango: Q3.12 es más preciso pero llega solo a ±8, mientras que Q7.8 llega a ±128 con menor precisión. Para sumar y restar hay que llevar los números al mismo formato con desplazamientos de bits; el error aparece al descartar bits, no al sumar.

Al multiplicar, los bits fraccionarios se suman (12 + 8 = 20) y el resultado necesita 32 bits; para volver a Q7.8 se desplaza 12 lugares a la derecha. En la división, el formato del resultado se determina buscando la mayor cantidad de bits fraccionarios que todavía permita que el cociente entre en 16 bits.

En la LC-3, que no tiene desplazamientos, multiplicación ni acarreo, las mismas operaciones se lograron con sumas, bucles y la partición del resultado en dos mitades de 15 bits. Los resultados coincidieron con los de C, y las pruebas adicionales con negativos, valores pequeños y desbordes dieron los resultados esperados.

La IA ayudó a obtener una base de trabajo, pero su resultado debe ser entendido, probado y depurado antes de aceptarlo, especialmente en assembly.

5. REFERENCIAS BIBLIOGRÁFICAS

[1] Y. N. Patt and S. J. Patel, Introduction to Computing Systems: From Bits and Gates to C and Beyond, 2nd ed.: McGraw-Hill, 2004.

[2] S. Harris and D. Harris, Digital design and computer architecture: arm edition: Morgan Kaufmann, 2015.

[3] Universidad Nacional de Misiones, Facultad de Ingeniería, Arquitectura de Computadoras: Guía de Laboratorio N° 3a, Trabajar con Punto Fijo en el Procesador, 2026.